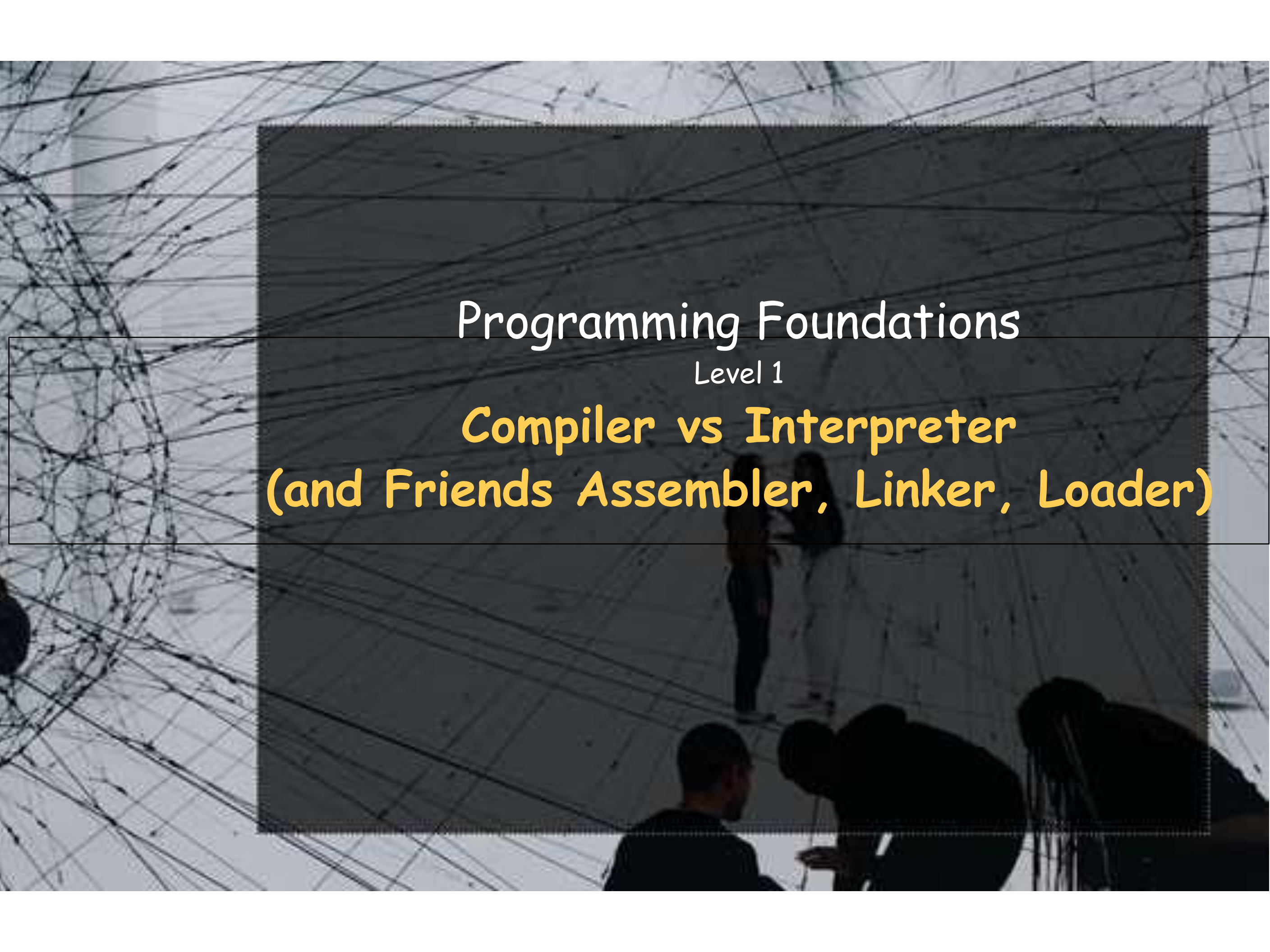




Programming Foundations Level 1



Programming Foundations

Level 1

Compiler vs Interpreter
(and Friends Assembler, Linker, Loader)



Programming Foundations

Level 1

Programming Languages and Levels

Compiler vs Interpreter (and Friends: Assembler, Linker, Loader)

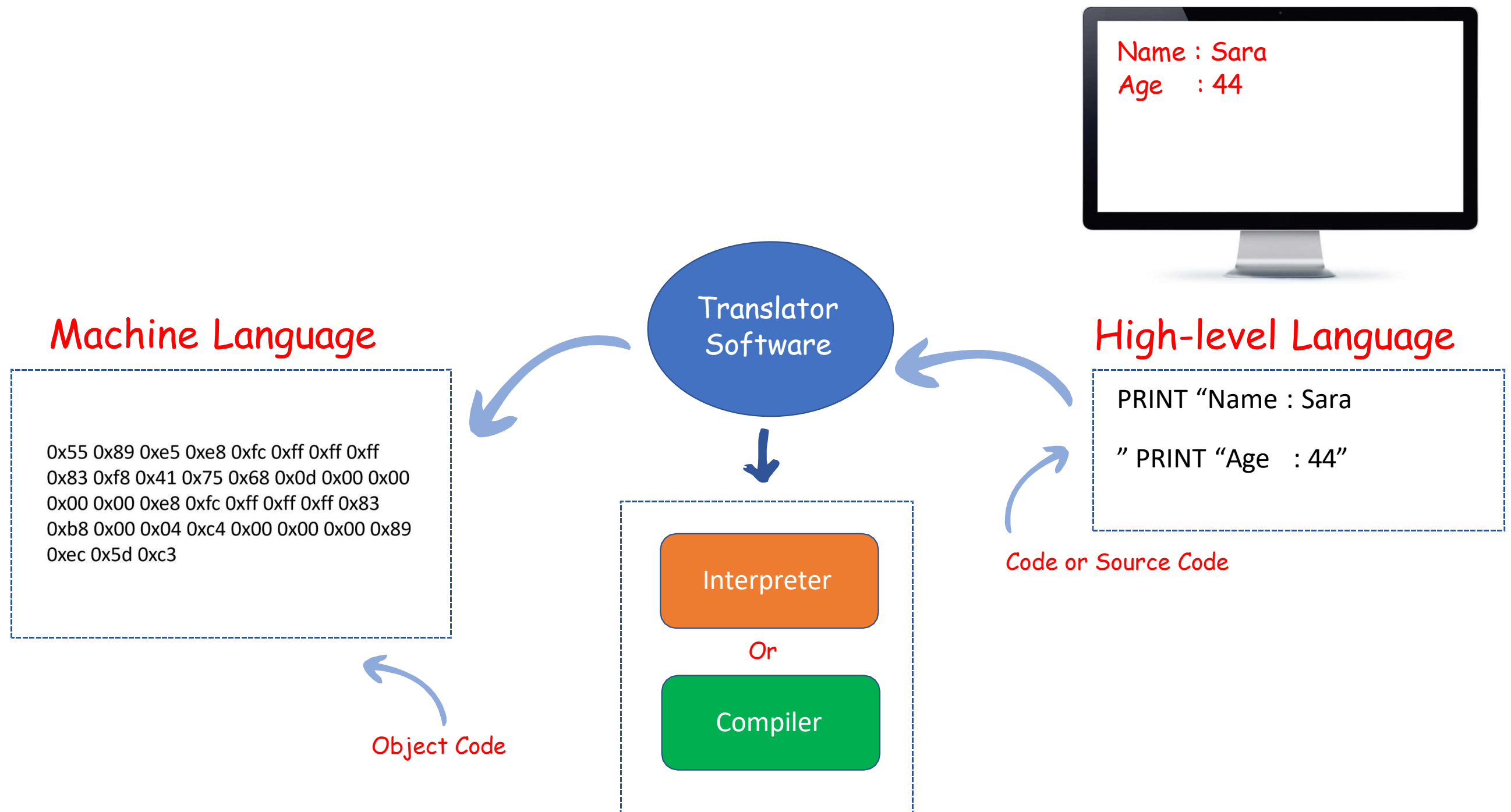
Computers only execute machine language (binary).

What we write is **source code** in human-friendly languages.

We therefore need **translator software** to bridge the gap: **compilers** (ahead-of-time), **interpreters** (line-by-line), and **assemblers** (assembly → machine).

We'll also see where **linkers**, **libraries**, **.exe files**, and **loaders** fit in the journey from **source to running program**.

You need a Translator!



We write high-level code (e.g., PRINT "Name : Sara").
CPU runs machine language.

A translator converts source code → object/machine code so the computer can execute it.

The background of the slide features a complex, abstract pattern of thin, dark lines crisscrossing over a lighter, textured surface. In the lower portion of the image, the silhouettes of several people are visible, appearing to be in a collaborative setting, possibly a meeting or a workshop, with some individuals leaning over a table or screen.

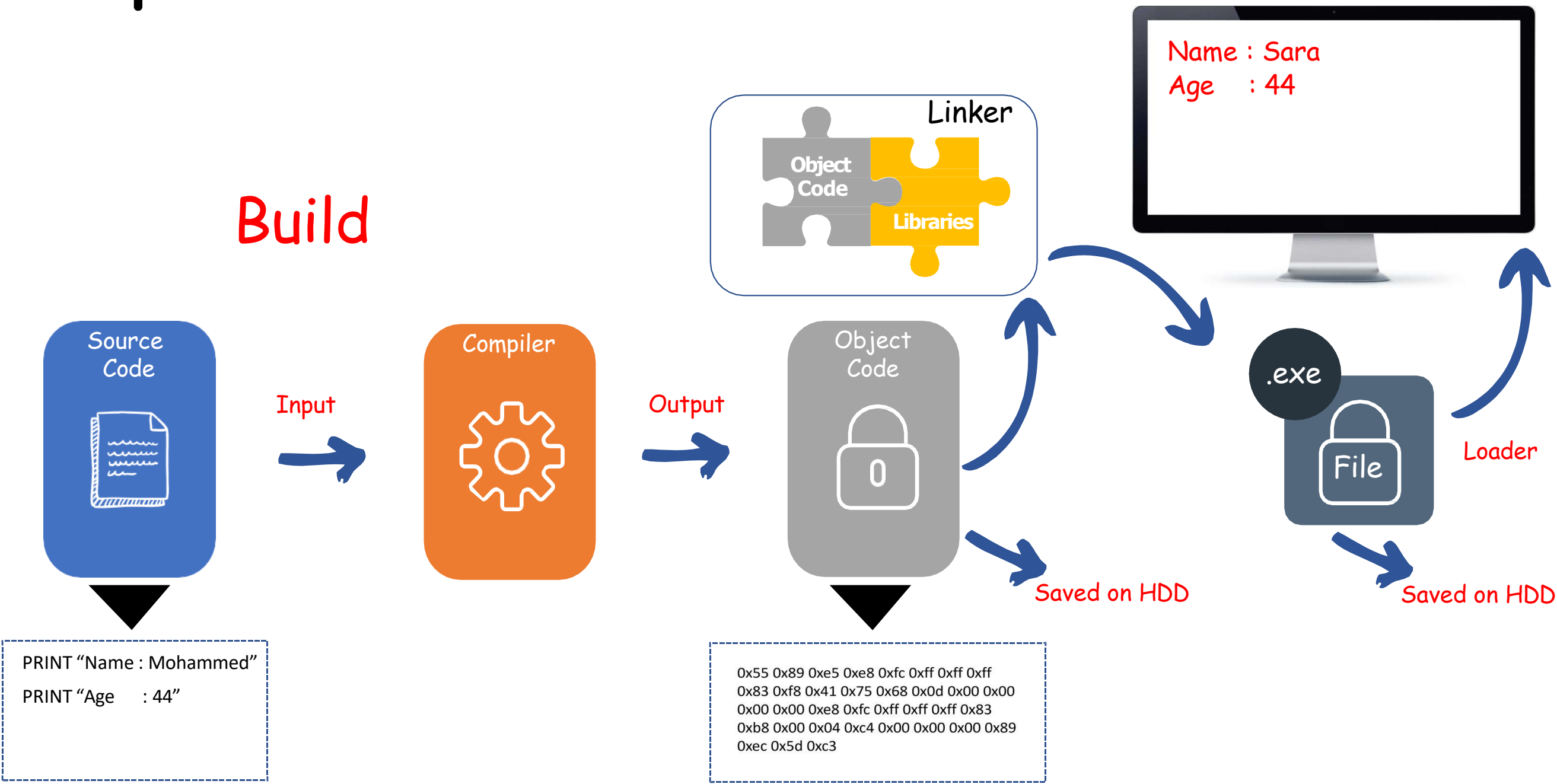
Programming Foundations

Level 1

Compiler

Compiler:

Build



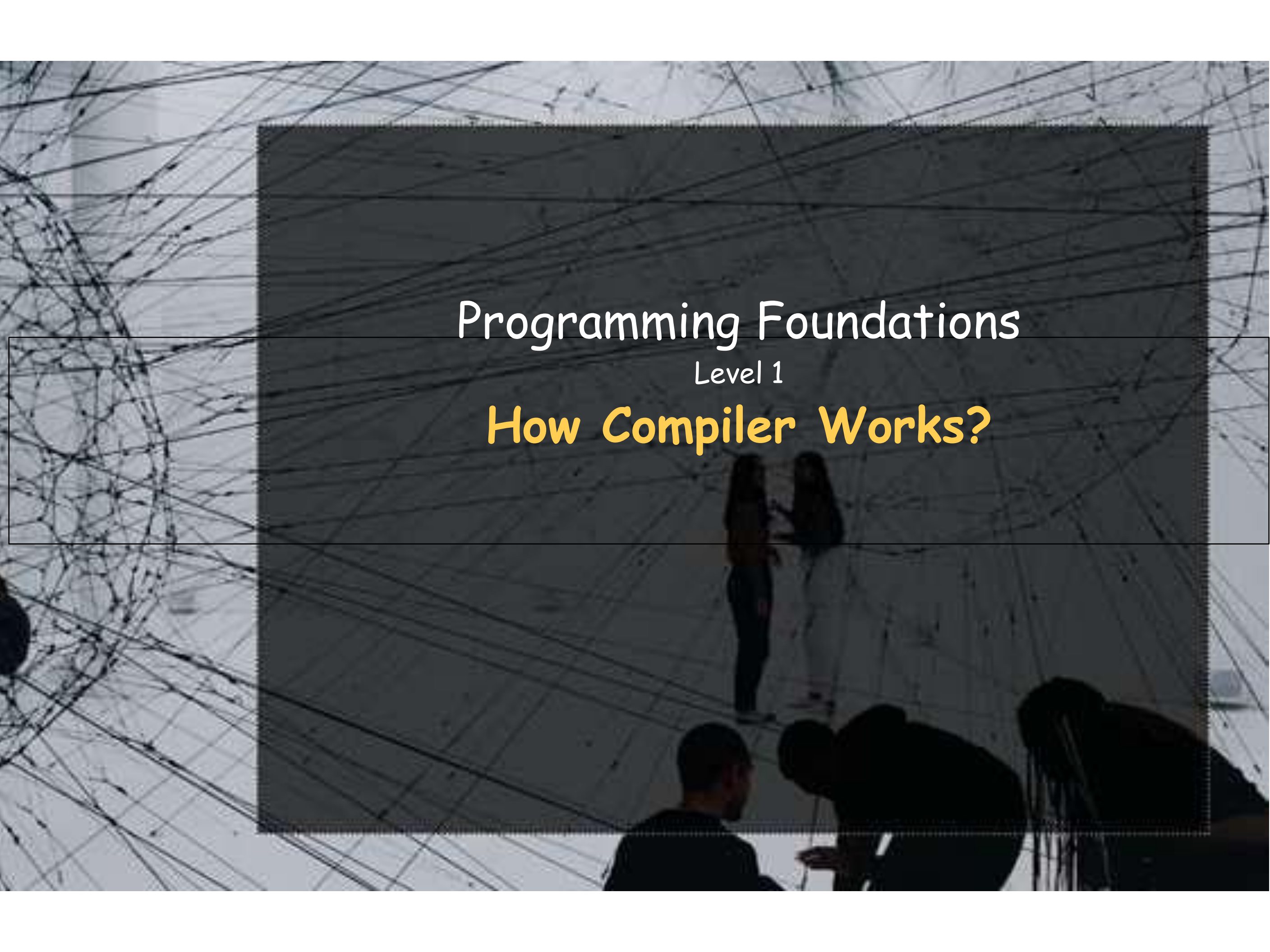
Compiler:

Input: Source code 🖱️ **Compiler** 🖱️ **Object code + Libraries** 🖱️ **Linker** 🖱️ **.exe** file (saved on disk) 🖱️ **Loader** loads it into memory 🖱️ **Program** runs and prints output (e.g., *Name : Mohammed, Age : 44*).

Compiler (build time): translates the **entire program** and reports **build errors** if found. If errors exist → **stop**; otherwise produce **object code**.

Linker: combines **object code** with **libraries** to create an **executable (.exe)**, stored on **HDD/SSD**.

Loader (run time): loads the **.exe** into memory and starts execution.
machine code is stored on disk, **.exe generated**, and **execution is fast**.



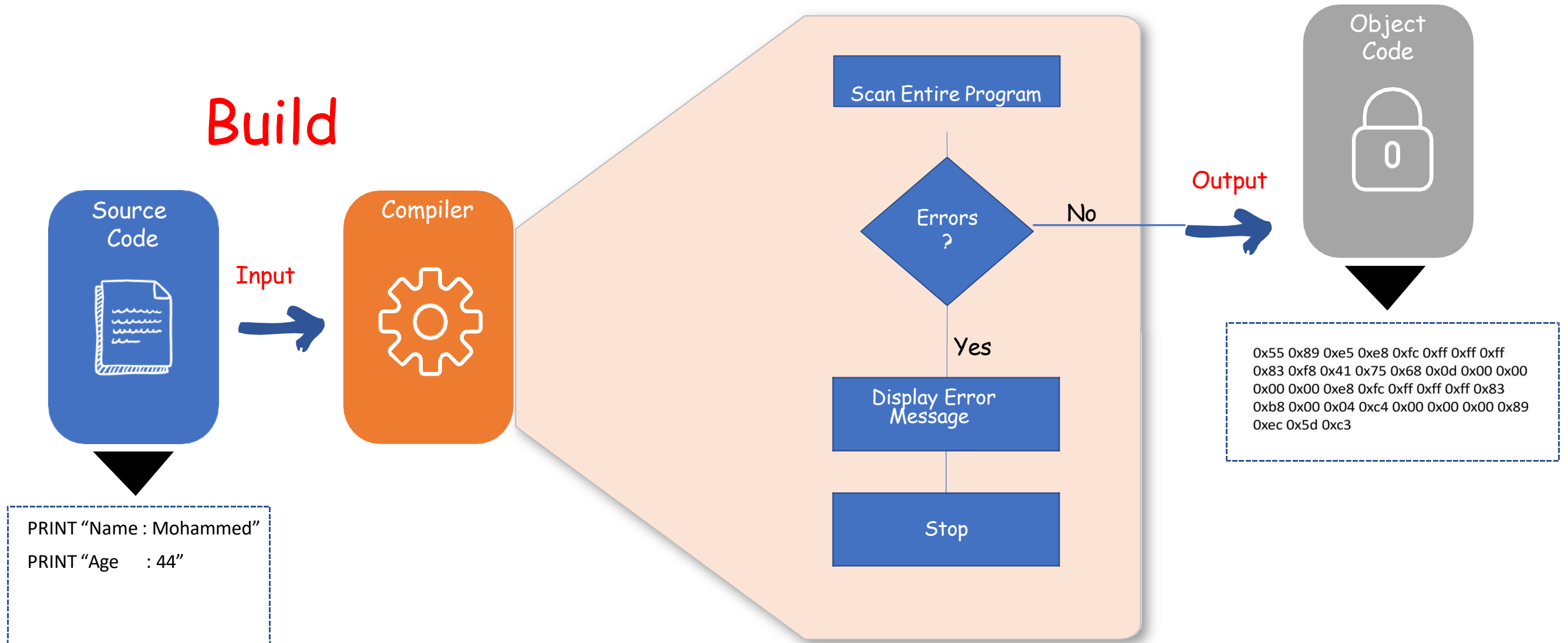
Programming Foundations

Level 1

How Compiler Works?

Compiler:

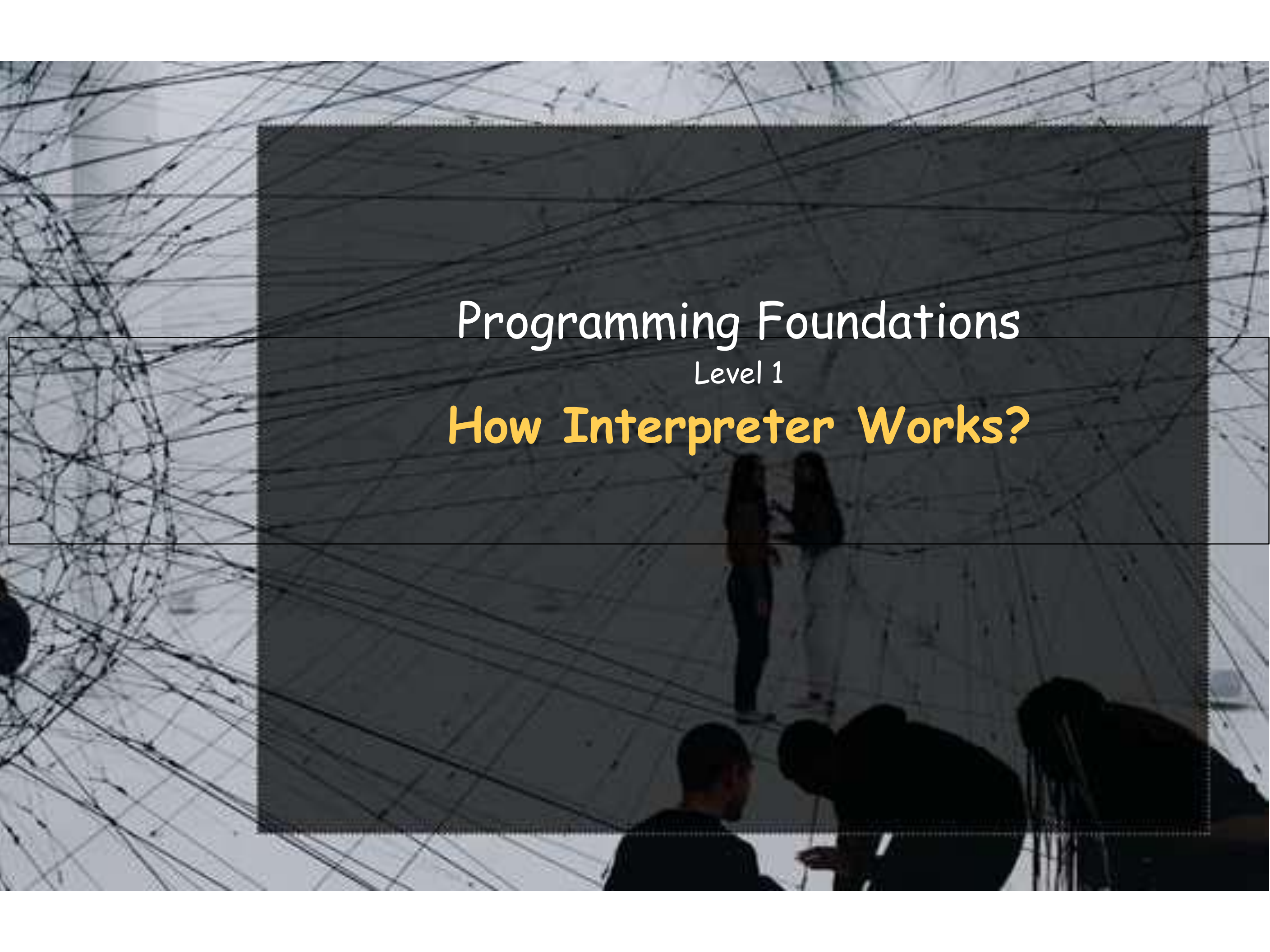
Build



Scan entire program.

If build errors? → display error message and stop.

If no errors → continue to produce object code and link.



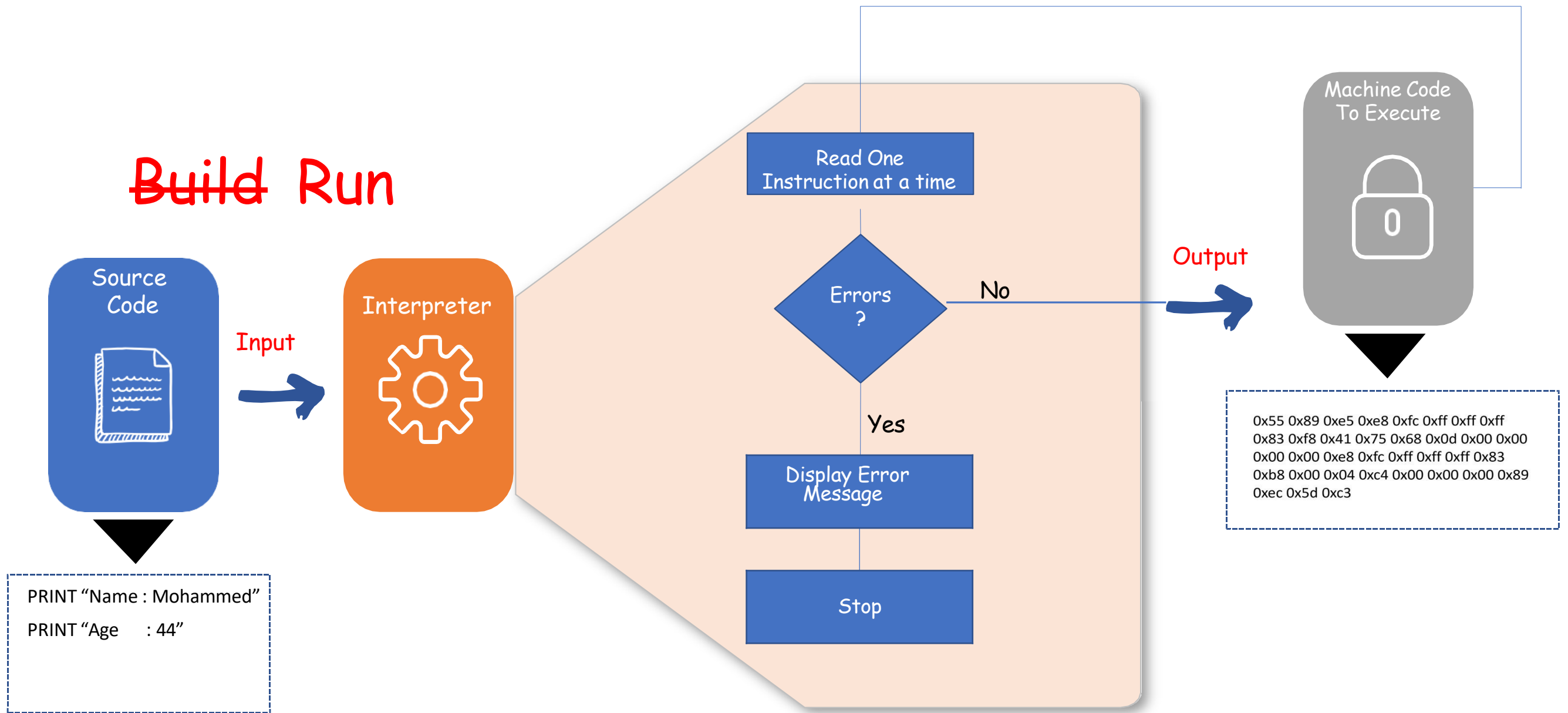
Programming Foundations

Level 1

How Interpreter Works?

Interpreter:

~~Build~~ Run

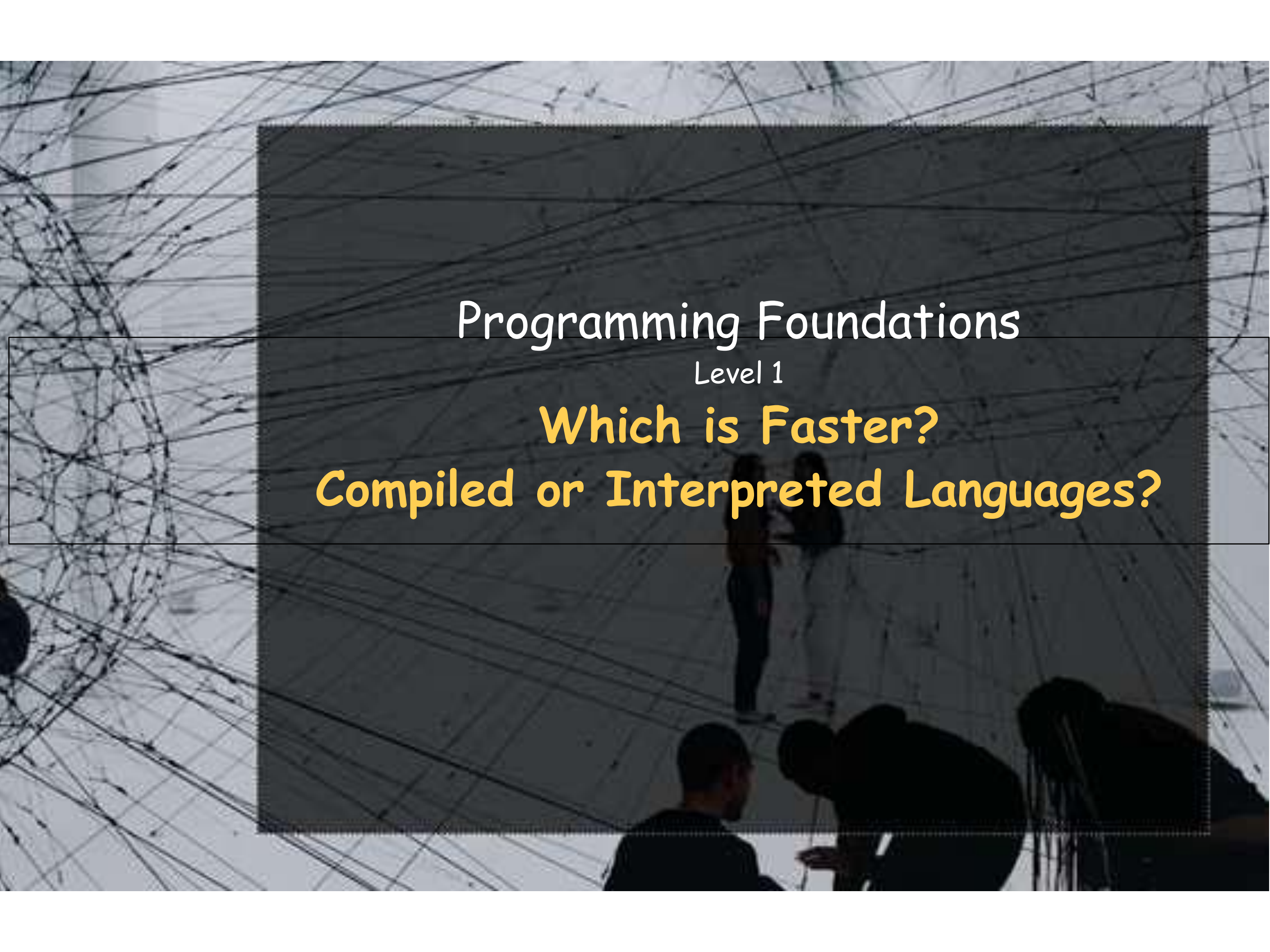


Reads one instruction at a time, translates it to machine code on the fly, then executes immediately.

If an error occurs → display error message and stop at that line.

No .exe is produced; no machine code is saved on disk.

Execution is generally slower.

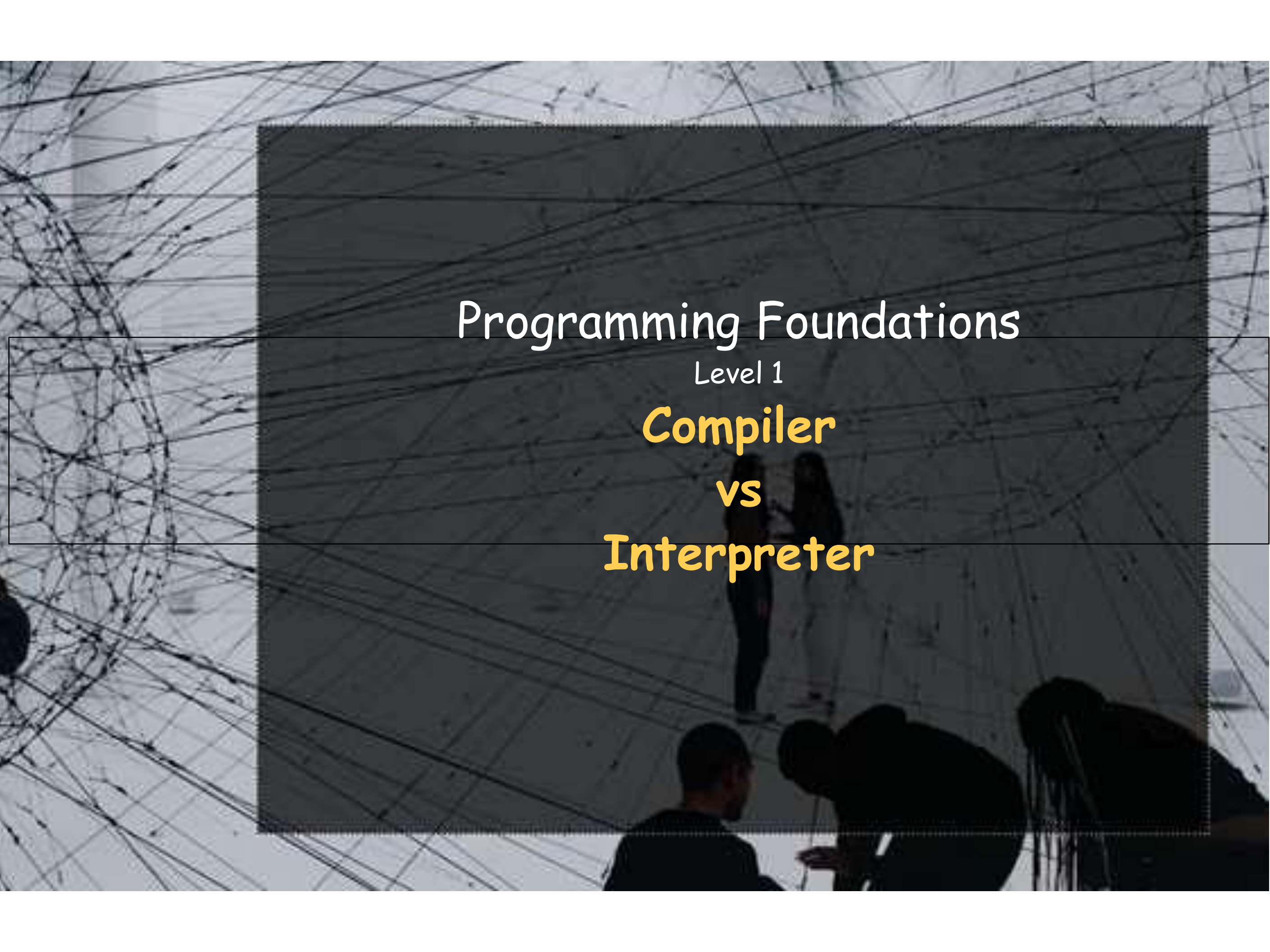


Programming Foundations

Level 1

**Which is Faster?
Compiled or Interpreted Languages?**

Compiled Languages are much faster than
interpreted Languages.



Programming Foundations

Level 1

Compiler vs Interpreter

Compiled

VS

Interpreted

Reads the Entire Code at once

Reads one instruction at a time

Execute the process is much faster

Execute the process is much slower

Machine Code is stored in HDD

Does not store any
machine code on HDD

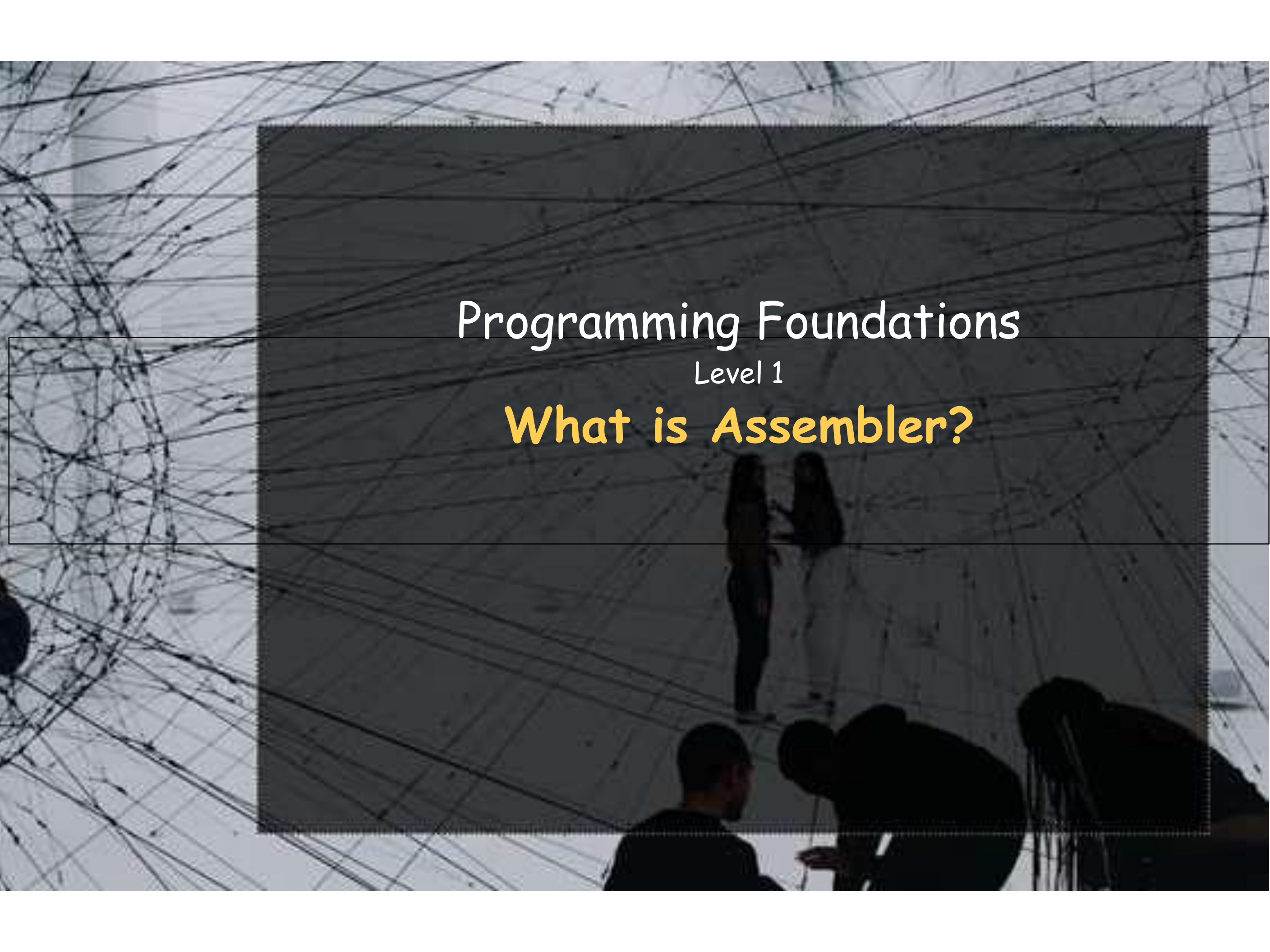
Generates exe file

No exe file generated

C, C++

Ruby, Python, Javascript.

Differences



Programming Foundations

Level 1

What is Assembler?

Assembler is the program that translate the assembly code to Machine code.

Interconnection

- ◆ **High-level source** needs a **translator** to become **machine code**.
- ◆ **Compiler path:** Source → Object → (Linker + Libraries) → **.exe** → **Loader** → Run. **Fast**, reusable binary.
- ◆ **Interpreter path:** **Line-by-line** translate & run. **No .exe**, typically **slower**.

By mastering these flows, learners can choose the right **toolchain** and predict **build/run behavior** with confidence. 

Activity

Instructions:

Answer the following questions based on what you learned about **Compilers, Interpreters, and Assemblers**.

Write your answers clearly and concisely.

- 1 What is a **Compiler**, and how does it work?
- 2 What is an **Interpreter**, and how does it work?
- 3 What is an **Assembler**?
- 4 What is a **Linker**?
- 5 What is an **.exe** file?
- 6 What is a **Loader**?
- 7 Which is faster — **Compiled** or **Interpreted** languages?
- 8 Does an **Interpreter** produce an **.exe** file?
- 9 Does an **Interpreter** save **machine code**?
- 10 Does a **Compiler** produce an **.exe** file?
- 1 1 Does a **Compiler** save **machine code**?



Activity Solution



Solution:

1 Compiler:

A program that translates the **entire source code** into **object code** (machine code) before execution.

It checks for **errors** during the build stage; if none exist, it produces an **.exe file** through linking.

2 Interpreter:

A program that **reads and executes source code line-by-line**.

If it finds an error, it stops immediately and shows an error message.

It does **not** create or save any **.exe file**.

3 Assembler:

A translator that converts **assembly language instructions** into **machine language (binary)**.

4 Linker:

A tool that **combines object code** and **required libraries** to produce an **executable (.exe) file**.

5 .exe file:

A file containing **machine code instructions** ready to be **loaded and executed** by the CPU

6 Loader:

A system program that loads the executable (.exe) into main memory (RAM) so the CPU can run it.

7 Which is faster:

Compiled languages are faster because their code is translated to machine code in advance, not during execution.

8 Does Interpreter produce .exe?

✗ No, an interpreter does not generate or store an .exe file.

9 Does Interpreter save machine code?

✗ No, it executes directly in memory without saving machine code on disk.

10 Does Compiler produce .exe?

✓ Yes, after translation and linking, a compiler produces an .exe file stored on disk.

1 1 Does Compiler save machine code?

✓ Yes, it saves machine code as part of the executable file for later use.

🏁 Conclusion

- ✓ **Compilers** translate whole programs, then **linkers** build **executables** that load and run natively—**fast**.
 - ✓ **Interpreters** translate and run **line-by-line**—**slower**, with **no .exe** saved.
 - ✓ **Assemblers** handle **assembly** → **machine**.
- Understanding these roles clarifies build vs run steps and helps you choose the right **language/toolchain** for performance and productivity.

Thank you 😊